

Scale to Zero and Live Migration of Stateful Apps on Kubernetes

Felicitas Pojtinger
Head of R&D
@pojntfx

{∞} Loophole Labs



What is Checkpoint/Restore?

CPU registers

Devices

Memory contents

File descriptors

Topics

1. Usecases for C/R
2. How C/R is implemented in Linux
3. Using C/R in Kubernetes with Architect

Scale to Zero



Regional websites

Fan out Varying demand
architecture

Checkpoint → Cluster usage goes down

Cluster usage goes down → Nodes released to AWS

Restore → Cluster usage goes up

Cluster usage goes up → Nodes requested from AWS

Live Migrations

Moving between nodes →
App has to restart →
downtime





Live migration between
nodes →
App state persists →
No downtime!





1. Live migrate to other node
2. Perform maintenance on node
3. Live migrate back to node



Checkpoint/Restore in Userspace

Mainlined kernel
patches

Kernel interfaces to
dump and restore
kernel resources

CRIU Checkpoint

1. Seizes all threads in the target process tree
2. Injects code snippets into target process ("infection")
3. Reads `/proc/pid/maps` to find out what memory the process allocates
4. Dumps memory contents from `/proc/pid/mem` etc. to disk, along with permissions, flags etc.
5. Goes through all file descriptors (files, sockets, pipes and special files) and dumps positions, flags, paths, connection state, buffers etc. to disk
6. Dumps namespace memberships (PID, network, mount, user etc.), CPU register state etc. to disk
7. Reads kernel state from `/proc` and `/sys` on the process and dumps it into the file
8. After checkpointing, process is either left running or killed

CRIU Restore

1. Recreates the process tree as a skeleton with fork
2. Locks and sets `/proc/sys/kernel/ns_last_pod`` to desired ``PID - 1`` to restore processes with the original PID)
3. Recreates or joins namespaces that were like they were before the checkpoint
4. Reopens files at original paths, recreates pipes, sockets and TCP connections
5. Restores in-flight data in pipes and buffers by manipulating kernel state
6. Remaps memory regions (and mmap'ed files) at the original addresses, then fills memory from the memory images
7. Restores CPU registers including the instruction pointer
8. Restores credentials, signal handlers etc.
9. Unpauses the restored process by jumping into the restored instruction pointer

No runtime overhead

Checkpoints are
(somewhat)
portable

Can fail for
complex/arbitrary
applications



Great security properties

Virtualization of syscalls, FS and
networking

gVisor Process Start

1. Reads the OCI runtime spec (`config.json`) just like any other OCI runtime would
2. Creates a new process for the sandbox and puts it into a network namespace, configures resources with cgroups
3. Starts gofer process (a filesystem process) with very limited capabilities (no `CAP_SYS_ADMIN` etc.)
4. Starts Sentry (the sandbox component) in the sandbox process we've started earlier
5. Sets up a gVisor kernel (task scheduler, signal handlers, VFS etc.)
6. Mounts emulated filesystems via the gofer (e.g. `/proc`, `/sys` and so on)
7. Sets up netstack (Go implementation of the kernel's network stack in userspace)
8. Sets up Linux seccomp filters on the sandbox (Sentry)
9. Loads the container's binary (PID 1) and sets up memory mappings, the stack, interpreter etc.
10. Creates an init task in the kernel (with memory, file descriptors, FS context, credentials and signal handlers)
11. Unblocks init task on startup

Syscalls → caught by
KVM/systrap, then emulated
in Sentry

FS I/O → caught by
VFS, resolved via
RPCs to Gofer

Network I/O → caught
by netstack, written
to AF_PACKET sockets

gVisor Checkpoint

1. Get a request via the sandbox control socket (sent by runsc checkpoint etc.) in the sandbox controller
2. Pauses all tasks by sending an internal SIGSTOP signal and waiting until all task goroutines have stopped
3. Stops all timers, waits for gofers and netstack to finish
4. Serialize gVisor kernel state like PIDs, CPU register states, credentials, IPC objects and namespaces
5. Serialize memory addresses and file descriptors, much like how CRIU does it, except using the virtualized resources, not the actual Linux kernel resources
6. Serialize filesystem state from the VFS, e.g. mount tables, host paths that correspond to the gofer paths
7. Serialize network state with netstack (TCP connection state, UNIX sockets etc.)
8. Serialize virtual device state (PTYs etc.)
9. Write into checkpoint image files based on protobuf schema
10. After checkpointing, process is either left running or killed & the sandbox torn down

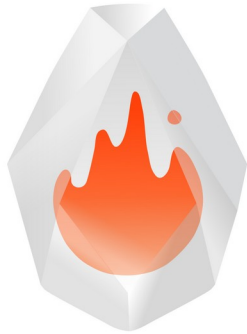
No need for PID emulation

No “infection” code

TCP
checkpoint/restore is
easier

gVisor Restore

1. Sandbox process, Sentry and gofer get started just like with the initial start, with minimal state
2. Checkpoint image files get read into memory, where the state decoder then parses the format and unmarshals into the relevant serializable gVisor components
3. Kernel state gets restored by iterating through tasks and creating goroutines for each, using the same internal PID as on checkpoint
4. CPU registers, signal handlers, credentials and FS context get set back in the gVisor resources
5. Memory managers get recreated in the same virtual address ranges as the checkpoint and memory pages get read from the checkpoint files
6. mmap-ed regions backed by an actual file get re-mapped to the same path on the host
7. File descriptor tables get restored for each task, by gofer re-opening them on the host or reading them from the checkpoint data for tmpfs
8. Network state gets restored by creating a new netstack instance and creating NICs and routing tables with the stored data from the checkpoint
9. TCP sockets, UNIX sockets etc. get restored in netstack
10. Devices (PTY etc.) get restored by recreating them based on the checkpoint data
11. To resume, the kernel clears the restore flag (similar to the stop flag)



Firecracker

Needs hardware acceleration
or nested virt

Starts lightweight microVMs,
not processes

KVM-based
isolation

Overhead because of
nested guest kernel

CPU templates make
CPUID masking easy

microVM Startup

1. Receives a REST API call that configures the internal VM configuration
2. Opens `/dev/kvm` and creates a VM file descriptor via `KVM_CREATE_VM`
3. Allocates guest memory via `mmap` and registers the new memory regions via the KVM devices
4. Creates vCPUs for each CPU via the KVM devices
5. Loads the Linux kernel into guest memory
6. Initializes virtio devices (disks/block devices, net, entropy etc.) and registers them on the VM bus
7. Configures the system for boot (setting up ACPI (x86)/Device Tree (aarch)) and spawns a (paused) thread for each vCPU
8. Applies seccomp filters to the VMMs and vCPU threads for better sandboxing (similar to how the Sentry is sandboxed in gVisor), then unpauses the vCPU threads and KVM begins executing the guest kernel

Code → Runs on physical CPU,
sandboxed via VT-d/VT-x

Device IO → virtio
devices

Networking → TAP
devices & netns

microVM Checkpoint

1. Receives a REST API call that pauses all vCPUs, which causes all vCPU threads to exit from KVM_RUN
2. Sends a SaveState event to each vCPU thread, which dumps CPU state (e.g. registers, MSRs, LAPIC state, TSC frequency, CPUID leaves etc.)
3. Dumps VM-level states (guest memory etc.) and virtio device state from the host (serializes queues and configuration, e.g. MAC address or TAP device names)
4. Serializes device state into a snapshot file and guest memory into a separate file
5. After checkpointing, VM is either left running (by resuming the vCPU threads) or stopped completely

Lazy-loading of memory

Firecracker controls the
entire CPU and devices

No need for TCP
restore

No need for a VFS

File descriptors,
syscalls etc. are just
memory

microVM Restore

1. Receives REST API call that opens the snapshot state file and deserializes it
2. Loads guest memory with mmap (so that it can be lazy-loaded), either from a file (for non-anonymous memory) or initialized with zeros
3. Reads vCPU counts, memory sizes, CPU templates etc. and configures KVM like at startup, just using the values from the checkpoint files
4. Restores vCPUs by setting TSC frequencies and registers for each vCPU from the checkpoint
5. Restores the Firecracker implementations of the virtio devices from the checkpoints by deserializing them, (re-)opening the TAP devices, setting the MAC addresses and recreates the queues
6. Like with the original startup process, it applies seccomp filters to the VMMs and vCPU threads for better sandboxing (similar to how the Sentry is sandboxed in gVisor), then unpauses the vCPU threads and KVM begins executing the guest kernel

PVM

Pageable-based Virtual Machine

VMs need a KVM vendor
implementation

Emulation is
slow

Nested virt has high
latency and security
issues

lore.kernel.org

linux-kernel.vger.kernel.org archive mirror

search help / color / mirror / Atom feed

* [RFC PATCH 00/73] KVM: x86/PVM: Introduce a new hypervisor
@ 2024-02-26 14:35 Lai Jiangshan
2024-02-26 14:35 ` [RFC PATCH 01/73] KVM: Documentation: Add the specification for PVM Lai Jiangshan
` (74 more replies)
0 siblings, 75 replies; 82+ messages in thread
From: Lai Jiangshan @ 2024-02-26 14:35 UTC (permalink / raw)
To: linux-kernel
Cc: Lai Jiangshan, Linus Torvalds, Peter Zijlstra,
Sean Christopherson, Thomas Gleixner, Borislav Petkov,
Ingo Molnar, kvm, Paolo Bonzini, x86, Kees Cook, Juergen Gross,
Hou Wenlong

From: Lai Jiangshan <jiangshan.ljs@antgroup.com>

This RFC series proposes a new virtualization framework built upon the KVM hypervisor that does not require hardware-assisted virtualization techniques. PVM (Pagetable-based virtual machine) is implemented as a new vendor for KVM x86, which is compatible with the KVM virtualization software stack, such as Kata Containers, a secure container technique in a cloud-native environment.

The work also led to a paper being accepted at SOSP 2023 [sosp-2023-acm] [sosp-2023-pdf], and Lai delivered a presentation at the symposium in Germany in October 2023 [sosp-2023-slides]:

PVM: Efficient Shadow Paging for Deploying Secure Containers in Cloud-native Environment

PVM has been adopted by Alibaba Cloud and Ant Group in production to host tens of thousands of secure containers daily, and it has also been adopted by the Openanolis community.

Motivation
=====
A team in Ant Group, co-creator of Kata Containers along with Intel, deploy the VM-based containers in our public cloud VM to satisfy dynamic resource requests and various needs to isolate workloads. However, for safety, nested virtualization is disabled in the L0 hypervisor, so we cannot use KVM directly. Additionally, the current nested architecture involves complex and expensive transitions between the L0 hypervisor and L1 hypervisor.

So the over-arching goals of PVM are to completely decouple secure container hosting from the host hypervisor and hardware virtualization support to:
1) enable nested virtualization within any IaaS clouds without affecting the security, flexibility, and complexity of the cloud platform;
2) avoid costly exits to the host hypervisor and devise efficient world switching mechanisms.

Why PVM
=====
The PVM hypervisor has the following features:

- Compatible with KVM ecosystems.

- No requirement for hardware assistance. Many cloud provider doesn't



PVM Internals

- **Guest kernel** runs in Ring 3 like userspace; guest/host separation enforced via software flags and shadow page tables
- **Guest syscalls** trap to in-kernel switcher, which updates CR3 and GSBASE, then jumps directly to guest kernel handler via SYSRETQ (no hypervisor)
- **Guest hypercalls** use SYSCALL in supervisor mode, trap to hypervisor for PVM-specific operations (page table switching, TLB, MSR) or standard KVM hypercalls
- **Shadow page tables** map guest physical to host physical addresses; guest page table modifications intercepted and mirrored to shadow tables
- **Interrupts** trap to hypervisor for handling (shadow MMU, emulation) or injection back to guest

Very performant for CPU-
bound processes

Slow if page
table is used
heavily

Not mainlined yet



During migrations,
networking usually breaks

XDP can be used
for both egress
and ingress

XDP runs in driver
mode, so lots of
kernel steps can be
skipped

loopholelabs.io

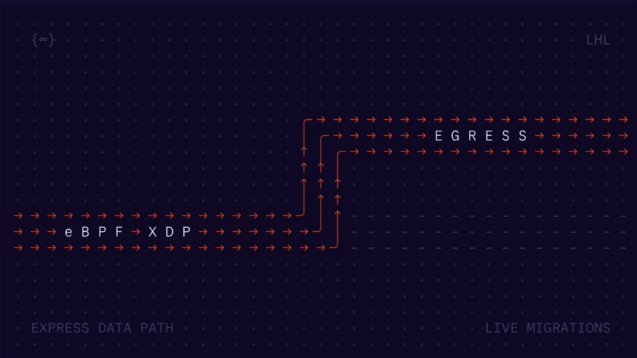
\$ // Heading to KubeCon North America 2025? // [Meet Us at Booth #1752](#) →

{∞} Loophole Labs

About / Blog / Join Waitlist →

~/ BLOG / ENGINEERING / USING XDP FOR EGRESS TRAFFIC (14 MINUTE READ)

An eBPF Loophole: Using XDP for Egress Traffic



TL;DR:

XDP (eXpress Data Path) is the fastest packet processing framework in Linux - but it only works for incoming (ingress) traffic. We discovered how to use it for outgoing (egress) traffic by exploiting a loophole in how the Linux kernel determines packet direction. Our technique delivers 10x better performance than current solutions, works with existing



Author:
Alex Sørli
Founding Engineer

Published:
2025.11.04

Topic(s):
Engineering

Socials:
[/in/ alex-sorlie](#) →

On this page:

- The Line-Rate Problem
- XDP's Main Limitation
- Why Traffic Control Isn't Good Enough



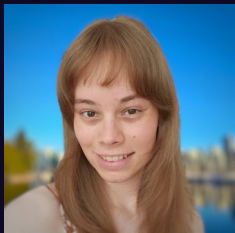
Network migration via eBPF XDP

1. Intercept traffic
2. Buffer/pause traffic before a checkpoint
3. Do a checkpoint
4. Restore from checkpoint on different (migration) or same (scale to zero) node
5. Flush buffers
6. Unpause connection

Demo

Checkpoint/Restore in Kubernetes with Architect

/~\ Architect



Felicitas Pojtinger

Fediverse:

Bluesky:

GitHub:

LinkedIn:

Web:

@pojntfx@mastodon.social

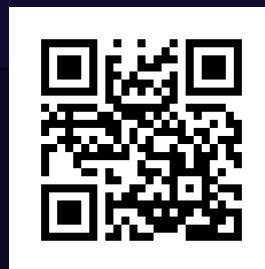
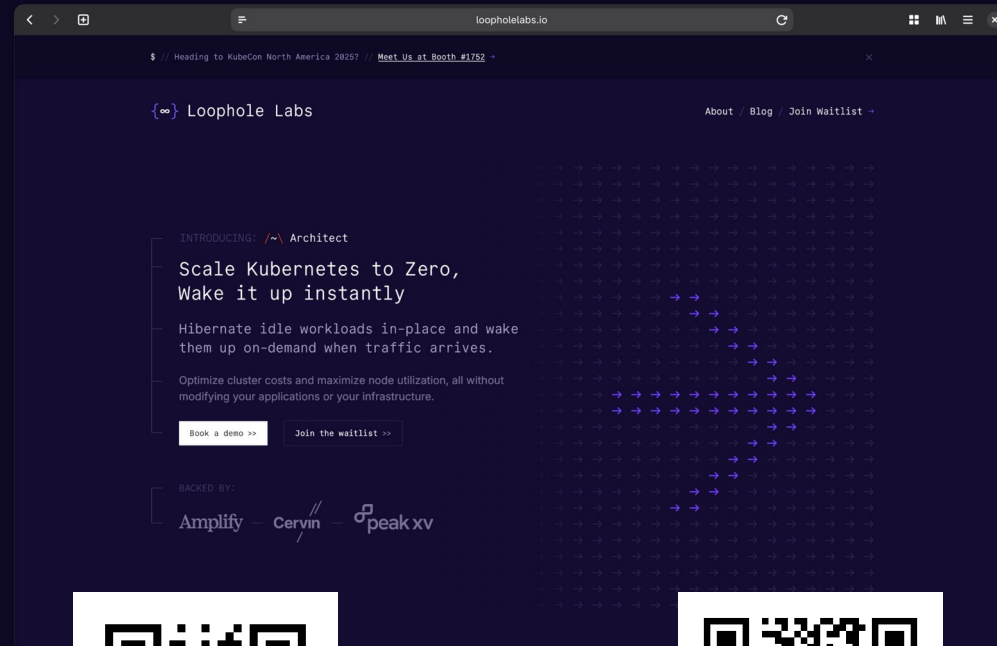
@felicitas.pojtinger.com

@pojntfx

in/pojntfx

<https://felicitas.pojtinger.com/>

{∞} Loophole Labs



← Waitlist

Slides & Links →

